



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	Dzikri Cahayadi Alamsyah
NPM	2428240126
Kelas	SI5C
Topik Tugas 1	Musik (Topik 15)
Entitas utama	Lagu (songs)
Tautan repositori	https://github.com/DzikriMDP/si5c_kuis1_Dzikri
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	06/10/2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
1. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
2. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
3. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
4. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis2_SI5C_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Musik (Topik 15)
Entitas utama (tunggal)	Lagu (song)
Alamat dasar	/songs
Field data	id, judul, artis, album, tahunRilis, durasiDetik
Field wajib saat POST	judul, artis, tahunRilis (album dan durasiDetik opsional)
Query string untuk filter	artis (contoh: /songs?artis=Senandung)
Data awal	Tiga lagu: Langit Hingga (Senandung, 2024), Malam Berdua (Senandung, 2023), dan Pulang (Ruang Senja, 2022)
Modul lain (jika ada)	Tidak ada modul lain. Terdapat route dasar GET / yang menampilkan identitas dan daftar endpoint.

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/songs	Ya	200	Ambil semua data (dengan filter bila ada)
2	GET	/songs/:id	Ya	200	Ambil satu data
3	POST	/songs	Ya	201	Tambah data
4	PUT	/songs/:id	Ya	200	Ubah data
5	DELETE	/songs/:id	Ya	200	Hapus data
6	GET	/	Tidak	200	Tampilkan identitas dan daftar endpoint

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.

```
C:\Repository Github\si5c_kuis1_Dzikri>npm i express
up to date, audited 95 packages in 14s

33 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

C:\Repository Github\si5c_kuis1_Dzikri>npm i dotenv
added 1 package, and audited 96 packages in 2s

34 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

C:\Repository Github\si5c_kuis1_Dzikri>npm i cors
added 2 packages, and audited 98 packages in 2s

35 packages are looking for funding
  run `npm fund` for details

3 high severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.

C:\Repository Github\si5c_kuis1_Dzikri>npm run dev
```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Jelaskan fungsi express, cors, dan dotenv, serta mengapa versi Node.js perlu memenuhi syarat Express 5.

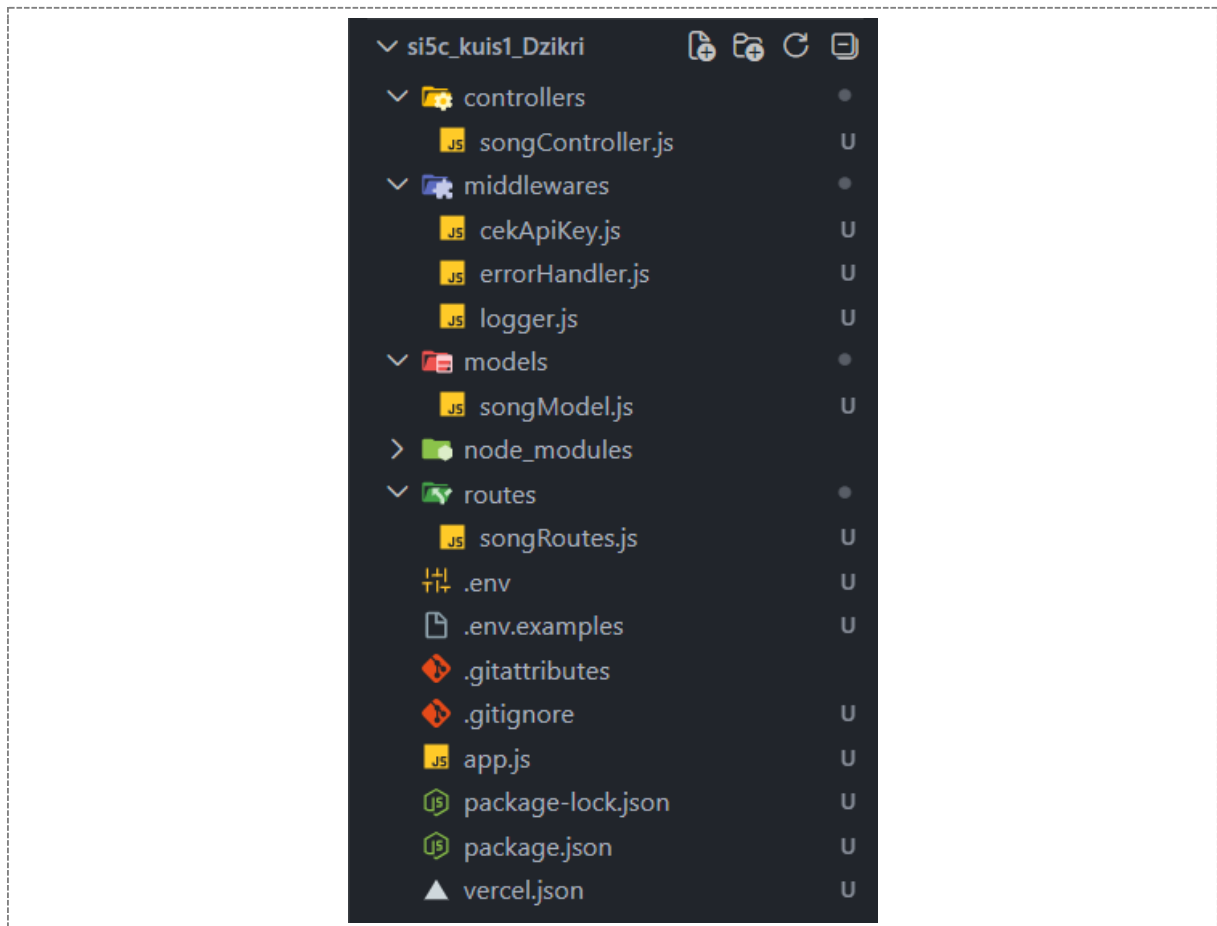
express adalah framework web untuk Node.js yang saya pakai untuk membuat server dan routing API. cors mengatur asal (origin) yang boleh mengakses API; pada app.js nilainya diambil dari CORS_ORIGIN dan hanya metode GET, POST, PUT, dan DELETE yang diizinkan. dotenv membaca file .env ke process.env, sehingga PORT, API_KEY, dan CORS_ORIGIN tidak ditulis langsung di dalam kode. Express 5 (package.json memakai ^5.2.1) membutuhkan Node.js versi 18 atau lebih baru, jadi versi yang tampil pada perintah node -v harus memenuhi syarat tersebut.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Jelaskan tugas setiap folder dalam satu kalimat.

routes berisi pemetaan metode dan alamat ke fungsi controller (songRoutes.js). controllers berisi logika tiap endpoint, yaitu validasi, pemanggilan model, dan pengiriman response (songController.js). models berisi data lagu beserta fungsi untuk mengelolanya (songModel.js). middlewares berisi fungsi yang berjalan sebelum atau sesudah route, yaitu logger.js, cekApiKey.js, dan errorHandler.js.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/songModel.js	Menyimpan array songs berisi tiga lagu awal dan variabel nextId yang dimulai dari 4. Fungsi getAll(artis) menyaring lagu berdasarkan artis tanpa membedakan huruf besar dan kecil, getByld mencari dengan find, create memberi id dari nextId++, update menimpa data dengan id tetap, dan remove menghapus dengan splice.
controllers/songController.js	Berisi getAll, getByld, create, update, dan remove. Controller mengubah id dengan Number(req.params.id), memeriksa body lewat validateSong, memanggil fungsi model, lalu mengirim response atau meneruskan error ke errorHandler dengan next(errorHttp(...)).
routes/songRoutes.js	Membuat express.Router() dan menghubungkan GET /, GET /:id, POST /, PUT /:id, dan DELETE /:id ke fungsi pada songController. Router ini dipasang di app.js pada path /songs.
middlewares/logger.js	Fungsi logger mencatat waktu (toISOString), metode, dan URL setiap request ke konsol, lalu memanggil next().
middlewares/cekApiKey.js	Membaca header x-api-key dan membandingkannya dengan process.env.API_KEY. Jika kosong atau berbeda, middleware membalas 401 dengan pesan JSON dan tidak memanggil next(); jika cocok, request diteruskan.
middlewares/errorHandler.js	Berisi errorHttp (membuat Error dengan properti status), notFoundHandler (404 untuk endpoint yang tidak ada), dan errorHandler (400 untuk JSON rusak, status dari err.status, dan 500 untuk error lain).
app.js	Memuat dotenv, memasang logger, cors, dan express.json(), membuat route dasar GET /, lalu memasang /songs dengan cekApiKey dan songRoutes, diikuti notFoundHandler dan errorHandler. Server dijalankan dengan app.listen pada PORT (default 3000) selain saat NODE_ENV production, dan app diekspor dengan module.exports.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.

```
C:\Repository Github\si5c_kuis1_Dzikri>npm run dev

> tugas1-restful-2428240126@1.0.0 dev
> nodemon app.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
◇ injected env (4) from .env
Server berjalan di http://localhost:3000
[2026-10-06T06:00:49.833Z] GET /
[2026-10-06T06:00:53.741Z] GET /songs
█
```

Gambar 3. Server berhasil dijalankan

Penjelasan:

Jelaskan dari mana nomor port berasal dan apa yang terjadi bila port sudah dipakai aplikasi lain.

Nomor port berasal dari `process.env.PORT`. Nilainya dibaca dari file `.env` (`PORT=4000`); jika `PORT` tidak ada, `app.js` memakai `3000` (`process.env.PORT || 3000`). Bila port sudah dipakai aplikasi lain, server gagal berjalan dan Node.js menampilkan error `EADDRINUSE`. Solusinya mengganti nilai `PORT` di `.env` atau menghentikan aplikasi yang memakai port tersebut.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

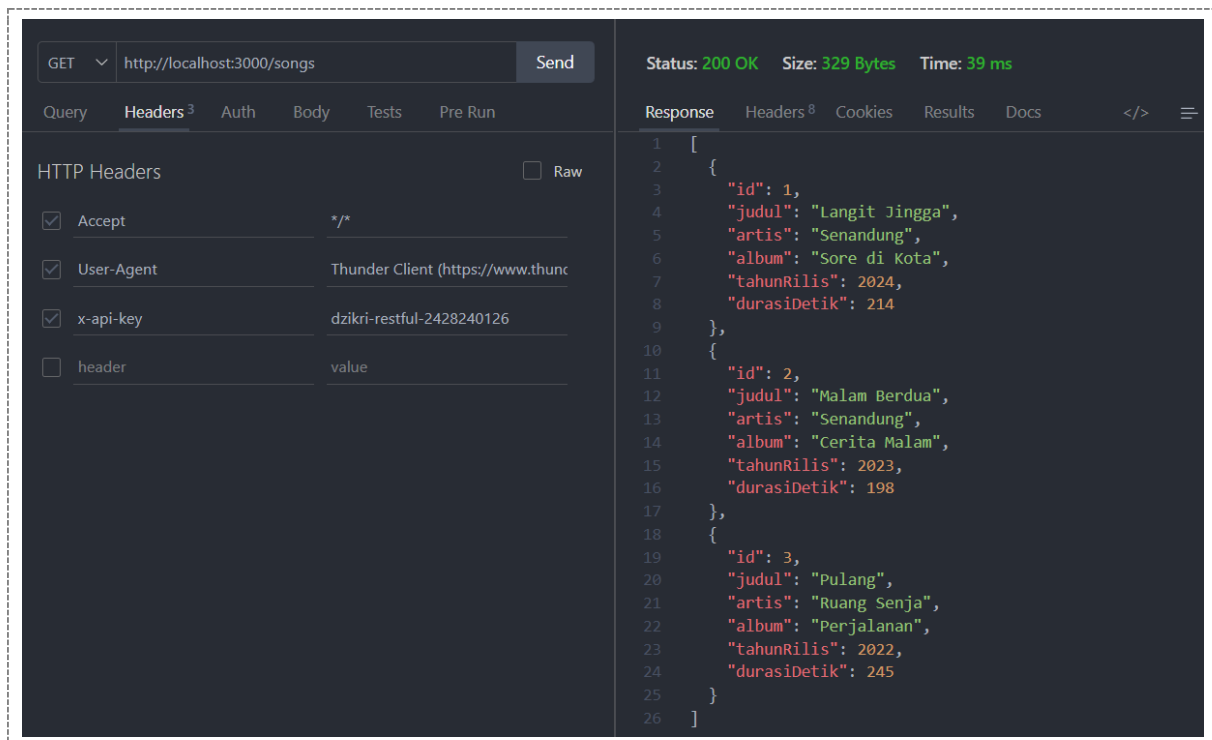
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /songs)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /songs
Header	x-api-key: dzikri-restful-2428240126
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK
Lulus?	☐ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

Penjelasan:

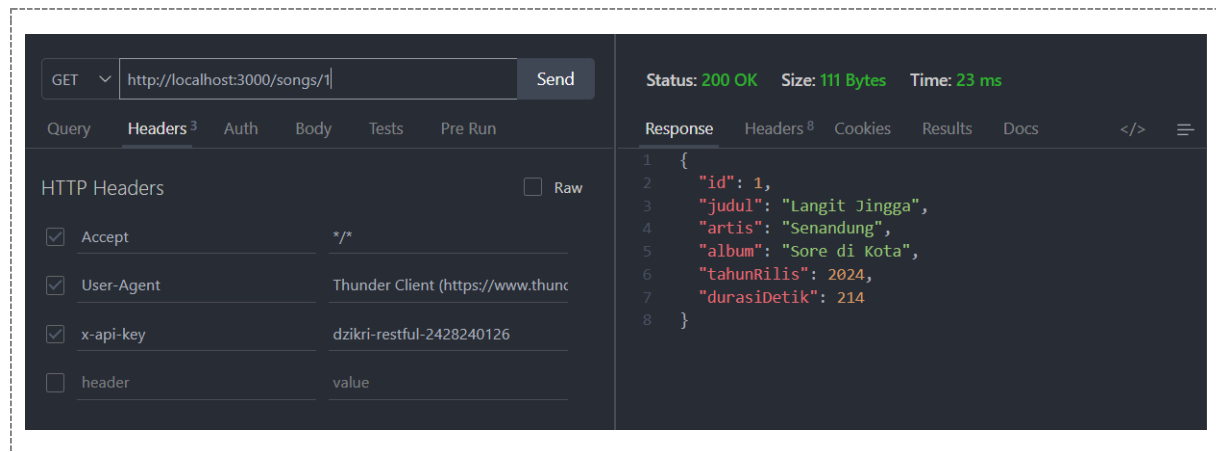
Jelaskan kode status yang muncul dan kemampuan endpoint ini. Sebutkan urutan pemanggilan: route mana, fungsi Controller mana, dan fungsi Model mana yang bekerja.

Endpoint ini mengembalikan status 200 beserta larik JSON seluruh lagu. Permintaan melewati logger, cors, express.json(), dan cekApiKey (semua route /songs memerlukan header x-api-key), lalu route router.get("/") memanggil songController.getAll. Controller membaca query artis dan memanggil songModel.getAll(artist); bila ?artis= diisi, hasilnya disaring berdasarkan nama artis.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /songs/1
Header	x-api-key: dzikri-restful-2428240126
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK
Lulus?	[] Ya [] Tidak



Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

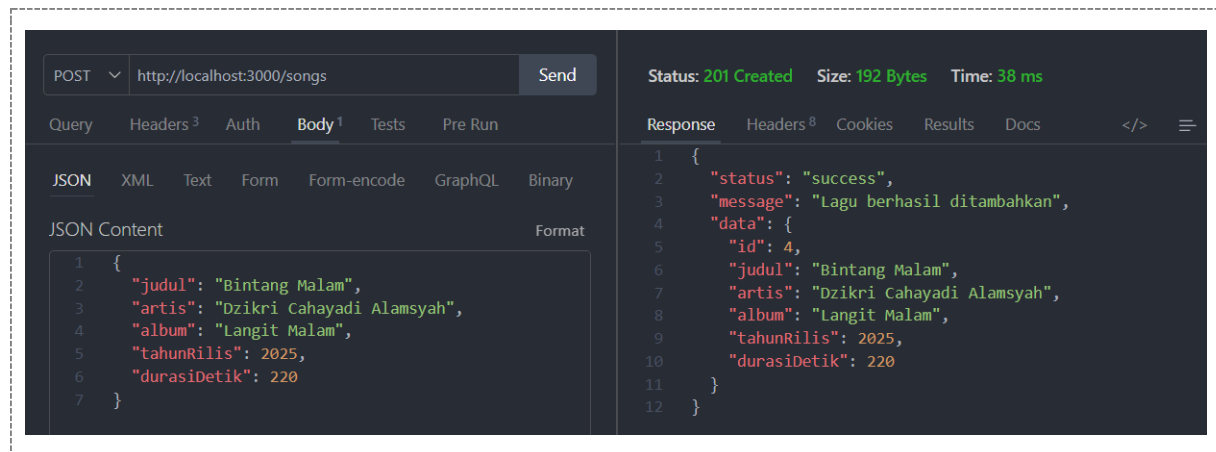
Jelaskan bagaimana ID dari alamat diambil (*req.params*) dan mengapa perlu diubah menjadi angka.

ID pada alamat diambil lewat *req.params.id*. Nilainya berupa teks, jadi diubah dengan *Number()* agar bisa dibandingkan dengan *song.id* yang bertipe angka pada *songModel.getById* (memakai *===*). Jika lagu ditemukan, controller mengirim satu objek lagu dengan status 200.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header *x-api-key* dan *Content-Type*, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /songs
Header	x-api-key: dzikri-restful-2428240126
Body	<pre>{ "judul": "Bintang Malam", "artis": "Dzikri Cahayadi Alamsyah", "album": "Langit Malam", "tahunRilis": 2025, "durasiDetik": 220 }</pre>
Hasil yang diharapkan	201 Created; JSON berisi status, message, dan data (objek lagu baru beserta id yang dibuat server).
Status hasil uji	201 Created
Lulus?	[] Ya [] Tidak



Gambar 6. Hasil POST data baru

Penjelasan:

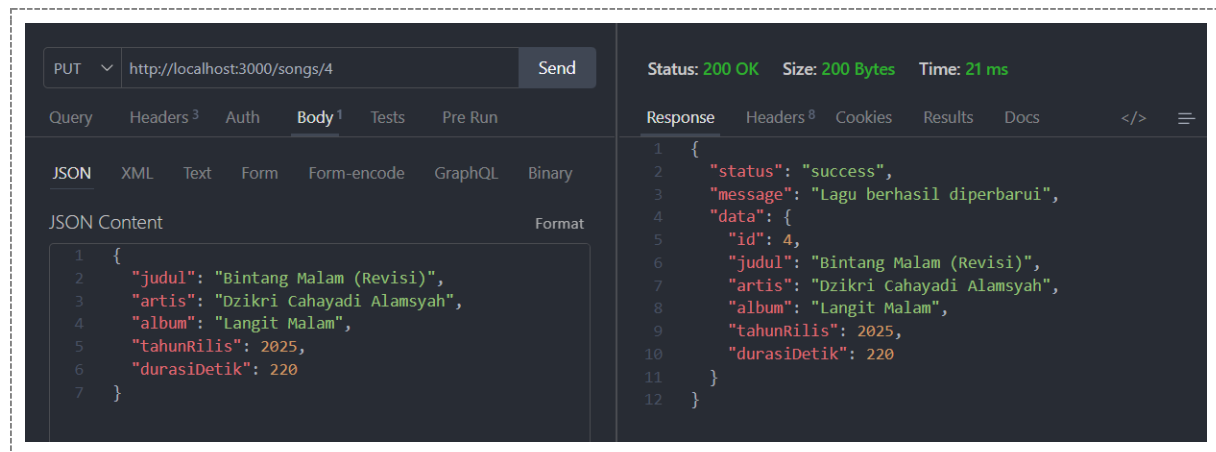
Jelaskan peran header `x-api-key` (middleware apa yang memeriksanya), peran `express.json()`, dan alasan status yang dipakai adalah 201, bukan 200.

Header `x-api-key` diperiksa oleh middleware `cekApiKey`, yang membandingkannya dengan `API_KEY` pada file `.env`. `express.json()` mengubah body JSON menjadi `req.body`, sehingga bisa divalidasi oleh `validateSong` dan dibaca controller. Status yang dipakai 201 Created karena request membuat data baru (id diambil dari `nextId`), bukan sekadar membaca data seperti pada status 200.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /songs/4
Header	x-api-key: dzikri-restful-2428240126
Body	<pre>{ "judul": "Bintang Malam (Revisi)", "artis": "Dzikri Cahayadi Alamsyah", "album": "Langit Malam", "tahunRilis": 2025, "durasiDetik": 220 }</pre>
Hasil yang diharapkan	200 OK; JSON berisi status, message, dan data (objek lagu yang sudah diperbarui).
Status hasil uji	200 OK
Lulus?	[] Ya [] Tidak



Gambar 7. Hasil PUT mengubah data

Penjelasan:

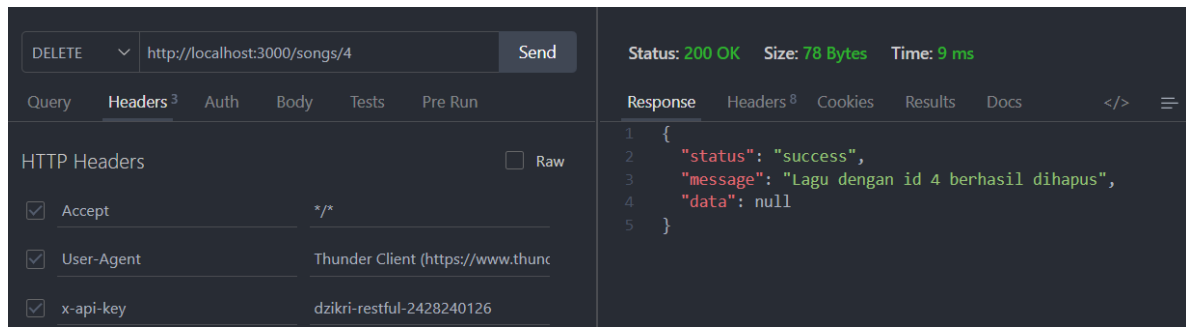
Jelaskan field apa yang berubah dan field apa yang tetap, serta layer mana yang melakukan perubahan.

Controller memeriksa dulu apakah id ada, lalu `validateSong` mewajibkan judul, artis, dan tahunRilis. Fungsi `songModel.update` menerima lagu dengan data dari body, sedangkan id tetap sama; field yang tidak dikirim (album atau durasiDetik) tidak muncul lagi pada hasil karena controller selalu meneruskan kelima field. Perubahan data dilakukan oleh layer Model, sedangkan Controller hanya memvalidasi dan menyusun response.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 200, dan respons JSON berisi pesan penghapusan.

Permintaan (request)	DELETE /songs/4
Header	x-api-key: dzikri-restful-2428240126
Body	(tidak ada)
Hasil yang diharapkan	200 OK; JSON berisi status, message (Lagu dengan id 4 berhasil dihapus), dan data null.
Status hasil uji	200 OK
Lulus?	[] Ya [] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Jelaskan arti status 204 dan mengapa respons tidak berisi apa pun.

Pada kode ini `songController.remove` mengembalikan status 200 OK dengan JSON berisi status, message, dan data null. Status 204 No Content berarti berhasil tanpa isi respons, sedangkan kode ini sengaja mengirim pesan penghapusan sehingga status 200 yang dipakai. Jika id tidak ditemukan, model mengembalikan false dan controller meneruskan error 404.

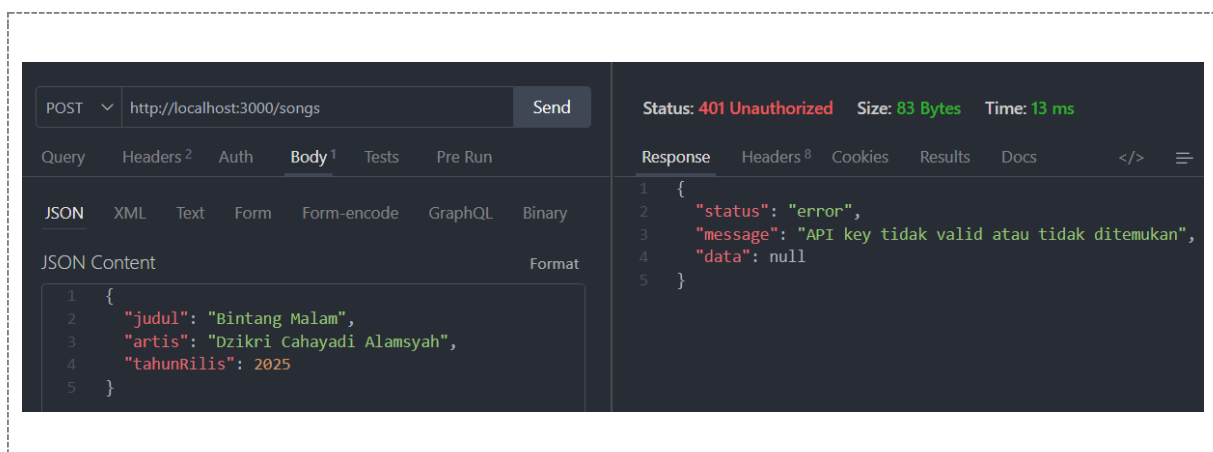
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /songs
Header	-
Body	{ "judul": "Bintang Malam", "artis": "Dzikri Cahayadi Alamsyah", "tahunRilis": 2025 }
Hasil yang diharapkan	401 Unauthorized; pesan JSON "API key tidak valid atau tidak ditemukan"; data tidak bertambah.
Status hasil uji	401 Unauthorized
Lulus?	[] Ya [] Tidak



Gambar 9. POST tanpa API key ditolak

Penjelasan:

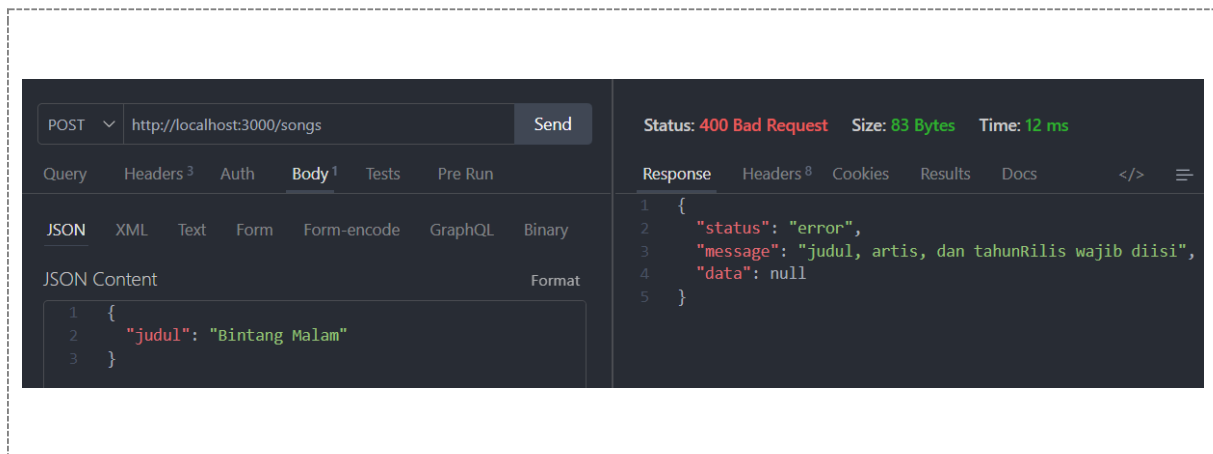
Jelaskan middleware mana yang menolak permintaan dan mengapa Controller tidak sempat dijalankan.

Middleware `cekApiKey` menolak permintaan karena header `x-api-key` tidak ada, dengan status 401 dan pesan "API key tidak valid atau tidak ditemukan". Middleware ini dipasang pada `app.use("/songs", cekApiKey, songRoutes)` sebelum router dan tidak memanggil `next()`, sehingga Controller tidak pernah dijalankan dan data tidak bertambah.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /songs
Header	x-api-key: dzikri-restful-2428240126
Body	{ "judul": "Bintang Malam" }
Hasil yang diharapkan	400 Bad Request; pesan "judul, artis, dan tahunRilis wajib diisi".
Status hasil uji	400 Bad Request
Lulus?	[] Ya [] Tidak



Gambar 10. POST dengan body tidak lengkap

Penjelasan:

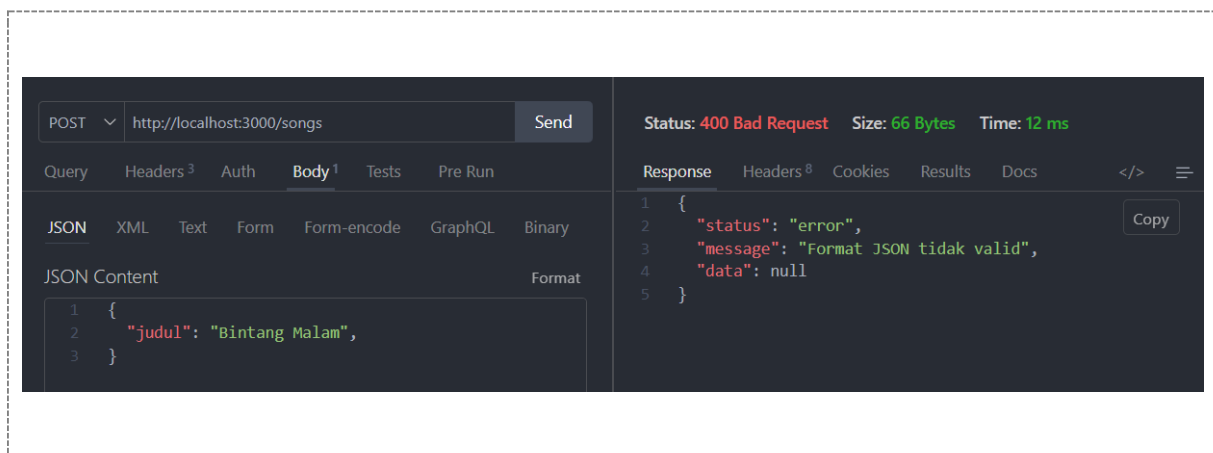
Jelaskan di layer mana validasi ini ditulis dan mengapa kode statusnya 400.

Validasi ditulis di layer Controller, pada fungsi `validateSong` di `songController.js`, yang mengembalikan pesan "judul, artis, dan tahunRilis wajib diisi". Controller lalu memanggil `next(errorHttp(400, ...))` dan errorHandler yang membalas. Statusnya 400 Bad Request karena kesalahan ada pada data yang dikirim klien.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /songs
Header	x-api-key: dzikri-restful-2428240126
Body	{ "judul": "Bintang Malam", }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request
Lulus?	[] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

Penjelasan:

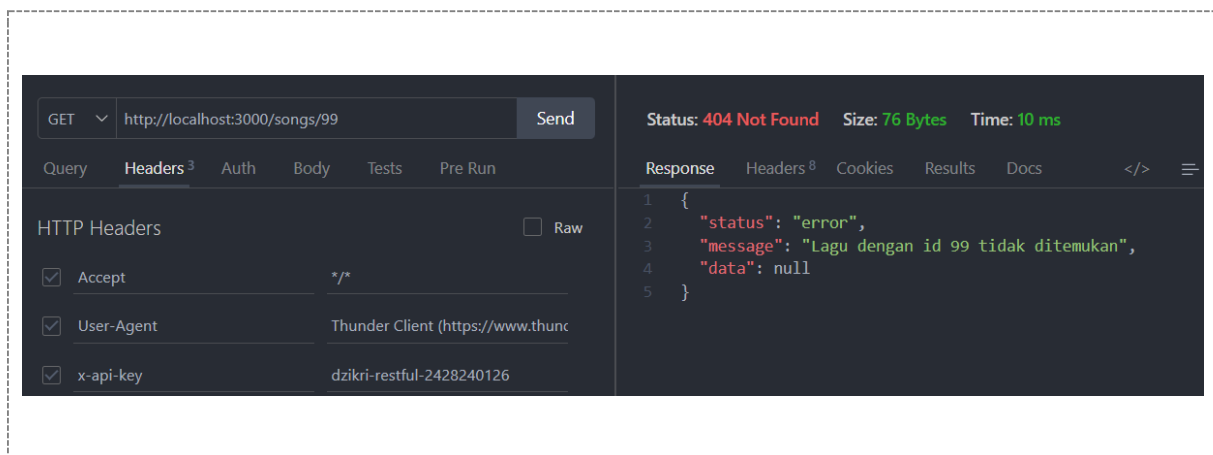
Jelaskan siapa yang mendeteksi JSON rusak ini dan bagaimana error handler mengubahnya menjadi respons yang rapi.

JSON rusak dideteksi oleh `express.json()` saat mengurai body dan melempar error bertipe `entity.parse.failed`. Error diteruskan ke `errorHandler` di `middlewares/errorHandler.js`, yang memeriksa `err.type` lalu membalas 400 dengan JSON "Format JSON tidak valid", sehingga klien tidak menerima halaman error HTML.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /songs/99
Header	x-api-key: dzikri-restful-2428240126
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON "Lagu dengan id 99 tidak ditemukan".
Status hasil uji	404 Not Found
Lulus?	[] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Jelaskan alur kode sampai pesan 404 dibuat dan siapa yang mengirimkannya.

Route memanggil `songController.getById`, yang mengubah id menjadi angka dan memanggil `songModel.getById(99)`. Karena hasilnya `undefined`, controller memanggil `next(errorHttp(404, "Lagu dengan id 99 tidak ditemukan"))`. Error itu diterima `errorHandler`, dan `errorHandler`-lah yang mengirim response 404 berformat JSON.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200		
T02	Menampilkan satu data berdasarkan ID	200		
T03	Menambah data baru	201		
T04	Mengubah data	200		
T05	Menghapus data	200		
N01	Menambah data tanpa API key	401		
N02	Menambah data dengan body tidak lengkap	400		
N03	Mengirim JSON yang rusak	400		
N04	Mengambil ID yang tidak ada	404		

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/DzikriMDP/si5c_kuis1_Dzikri
Pesan commit terakhir	Commit 1
Folder routes, controllers, models, middlewares ada	☐ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☐ Ya ☐ Tidak
.env.example ada	☐ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Tulis jawaban Anda di sini.

Route (routes/songRoutes.js) hanya mencocokkan metode dan alamat dengan fungsi controller, contohnya `router.get("/:id", songController.getById)`. Controller (controllers/songController.js) menangani request dan response, contohnya `songModel.getById(id)` lalu `res.json(song)` atau `next(errorHttp(404, ...))`. Model (models/songModel.js) mengelola data pada array, contohnya `songs.find((song) => song.id === id)`.

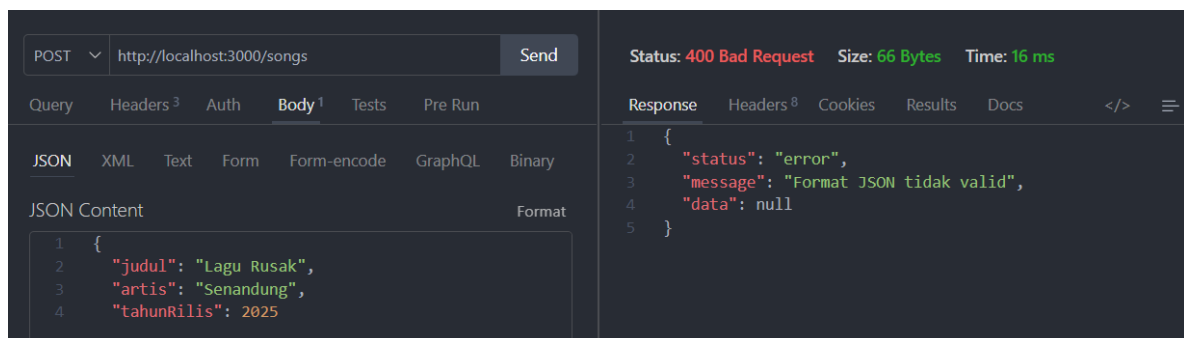
Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Tulis jawaban Anda di sini.

Dengan error handler terpusat (middlewares/errorHandler.js), controller cukup memanggil `next(errorHttp(404, ...))` tanpa menulis `res.status(404)` berulang, dan format error 400, 404, serta 500 menjadi seragam di satu tempat. Pada kode, error JSON rusak (N03) dan ID tidak ada (N04) sama-sama dibalas JSON berisi status "error", message, dan data null. Bukti dari screenshot pengujian:



Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	
4	Identitas saya (nama atau NIM) muncul pada data uji POST	
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	
6	Tabel rekapitulasi sesuai dengan tangkapan layar	
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100